

WIDS 2025 – Stock Market Prediction Using Reinforcement Learning

Mentor: Vaidehi Agrawal, Abhinav Badegaonkar

1 Introduction to the Course

Hola Amigos!

For the first two weeks of this course, you'll mainly be **reading and building foundational intuition**. This material is divided into two parts:

1. Reinforcement Learning (RL)
2. Finance

Reinforcement Learning (RL)

The PDF provided contains concise notes based on *Sutton and Barto*, the most widely recommended beginner friendly book on Reinforcement Learning. Throughout the course, we will refer to concepts, diagrams, and examples from this book.

We have attempted to present the essential ideas in a compact way. Because of this, the material may feel dense or difficult to understand on a first read. To help you, we have added links to simple explanations, YouTube videos, and articles wherever possible.

- Do not worry if you don't understand everything immediately.
- Feel free to search online, use LLMs, and revisit concepts multiple times.
- We do **not** expect anyone to fully grasp all topics on the first attempt.
- Make sure you **read examples** along the way, you'll mainly understand things through examples.

Here is the full book (free PDF): *Sutton and Barto – Reinforcement Learning: An Introduction*

Another major learning resource will be the legendary lecture series by Prof. David Silver. These lectures are free, well structured, and ideal for developing a deep intuition for RL.

Lecture playlist: *David Silver's RL Lectures*

2 Introduction to Reinforcement Learning

- Watch RL summary
- Watch lecture 1 by Prof David Silver

Reinforcement Learning (RL) studies how an **agent** should act in an **environment** so as to maximize a long-term numerical **reward**. Unlike supervised or unsupervised learning, RL does not provide a correct label or structure; instead, the agent learns through **trial-and-error interaction**. The feedback signal is scalar and may be delayed. The learner must discover which of its actions yield the most benefit by trying them.

RL problems consist of:

1. **States** (S): Observations describing the situation the agent is in.
2. **Actions** (A): Possible choices the agent can make.
3. **Rewards** (R): Numerical feedback received after acting.
4. **Policy** (π): A mapping from states to actions (deterministic or probabilistic). (state battery low results in action recharge)

Reinforcement Learning

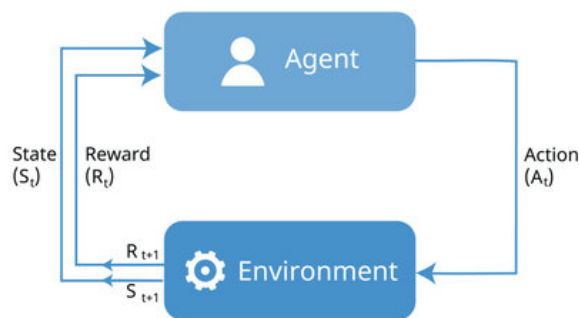


Figure 1: RL overview

In RL, the agent is not explicitly told which action is correct. Instead, it evaluates actions by interacting with the environment and observing the consequences. This is the central difference between RL and traditional supervised machine learning.

Example Consider a trading agent deciding whether to *Buy*, *Sell*, or *Hold* a given stock. A state S_t may contain recent price movements, volatility indicators, and current portfolio holdings. The chosen action A_t (*buy, hold or sell*) produces a portfolio change R_{t+1} (profit or loss). The agent repeats this process daily and, over many interactions, learns which actions maximize cumulative long-term return.

2.1 Agent–Environment Interaction

RL interaction proceeds in discrete time steps $t = 0, 1, 2, \dots$:

1. The agent observes the state S_t .
2. The agent selects an action A_t using its policy π .
3. The environment responds with a reward R_{t+1} and next state S_{t+1} .

The objective of RL is to find a policy π that maximizes the expected return, defined as a discounted sum of future rewards.

2.2 Return

For episodic tasks that eventually terminate:

$$G_t = R_{t+1} + R_{t+2} + \dots + R_T,$$

where T is the time step of termination.

For continuing tasks, typically a discount factor $\gamma \in [0, 1)$ is used:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

The discount factor γ expresses a preference for immediate rewards over distant ones and ensures mathematical convergence.

The discount factor becomes necessary as sometimes the agent may get stuck in a loop of states, $S1, S2, S3, S4, S1..$ and so on. Each cycle might result in a positive reward, which would set the total expected return from a state (sum of rewards without discounting) to be infinite.

2.3 Policies

A policy defines the agent’s behaviour. It may be:

- **Deterministic:** $\pi(s) = a$,
- **Stochastic:** $\pi(a|s) = P(A_t = a \mid S_t = s)$.

A good policy balances exploiting known information and exploring uncertain actions that may bring greater rewards.

2.4 Value Functions

Value functions evaluate how good it is to be in a given state or to take a given action in a given state. They express the notion of *long-term reward*.

The state-value function under a policy π :

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

The action-value function under π :

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

Value functions are fundamental because optimal solutions can be derived once they are known.

2.5 Exploration–Exploitation Tradeoff

Exploitation chooses actions that currently appear best based on accumulated experience.

Exploration tries uncertain actions to discover potentially better strategies.

Pure exploitation may get stuck in suboptimal behaviour; pure exploration wastes opportunities. Successful RL balances both themes.

2.6 Challenges in RL

- **Delayed rewards:** The effects of good or bad decisions may only appear later.
- **Credit assignment:** Determining which previous actions merit credit for eventual success.
- **Partial knowledge:** The agent typically does not know environment dynamics.

These challenges make trial-and-error learning essential and distinguish RL from supervised learning.

Additional Learning Resources

- Sutton and Barto 1.5: Tic-Tac-Toe learning agent.
- Prof David Silver’s lecture 1.

3 Multi-Armed Bandits (Chapter 2)

- You may choose to leave this chapter for now and complete it later after completing the project, as we are short on time.
- If you have done or plan to do CS747, you’ll have this.

The multi-armed bandit problem is the simplest setting in which the agent must choose between multiple actions, each with uncertain rewards, and learn over time which actions are better. It captures the fundamental **exploration–exploitation tradeoff** without involving states or long-term delayed effects.

A k -armed bandit has k actions (arms). At each time step t , the agent selects an arm $A_t \in \{1, 2, \dots, k\}$ and receives a scalar reward R_t . The goal is to maximize the expected sum of rewards over time:

$$\sum_{t=1}^T R_t.$$

There is no notion of state transitions or future consequences beyond the immediate reward. The challenge is to estimate which arms have the highest expected reward while still exploring uncertain arms.

Example (Trading Context)

Consider a scenario where each arm corresponds to a simple daily trading strategy (e.g., “buy-and-hold stock A”, “buy-and-hold stock B”, “hold cash”, “equal-weight portfolio”). Every trading day, the agent selects one strategy and observes the resulting daily return as reward. Over many days, the agent must learn which strategy has the highest expected return while occasionally exploring others, since market conditions and strategy performance may differ over time.

3.1 Action Values and Optimal Action

For each action a , define its **true value**:

$$q^*(a) = \mathbb{E}[R_t \mid A_t = a],$$

the expected reward when choosing a . These $q^*(a)$ are unknown and must be estimated from samples.

An **action-value method** maintains an estimate $Q_t(a)$ of $q^*(a)$ and uses it to select actions. Over time, as more rewards are observed, $Q_t(a)$ should converge to $q^*(a)$ if the method explores actions sufficiently.

3.2 Sample-Average Action-Value Estimation

A simple estimate of $q^*(a)$ is the sample average of the rewards obtained from that action:

$$Q_t(a) = \frac{1}{N_t(a)} \sum_{i=1}^{N_t(a)} R_i(a),$$

where $N_t(a)$ is the number of times action a has been selected up to time t , and $R_i(a)$ are the corresponding rewards.

Using sample averages is straightforward but requires storing counts (and possibly all past rewards, unless we use an incremental update) and is best suited for stationary problems where the reward distribution does not change over time.

3.3 Incremental Update Rule

The sample-average estimate can be updated incrementally without storing all past rewards. When action a is chosen at time t and reward R_t is obtained:

$$Q_{t+1}(a) = Q_t(a) + \frac{1}{N_t(a)}(R_t - Q_t(a)).$$

More generally, we may use a constant step-size parameter $\alpha \in (0, 1]$:

$$Q_{t+1}(a) = Q_t(a) + \alpha(R_t - Q_t(a)).$$

Here, $(R_t - Q_t(a))$ is the **error** between the observed reward and current estimate, and α controls how quickly the estimate adapts. Constant step-size is particularly useful in nonstationary environments.

3.4 Greedy and ε -Greedy Action Selection

A straightforward policy is the **greedy** policy:

$$A_t = \arg \max_a Q_t(a),$$

which always selects the action with the highest estimated value. However, pure greedy selection can get stuck: if an action is underestimated early, it may never be explored again.

To encourage exploration, the **ε -greedy** method is used:

- With probability $1 - \varepsilon$, choose a greedy action:

$$A_t = \arg \max_a Q_t(a).$$

- With probability ε , choose an action uniformly at random.

ε -greedy ensures that each action is tried infinitely often (for fixed $\varepsilon > 0$), which is important for converging to good estimates of $q^*(a)$.

Example (Trading Context)

Suppose the agent has $k = 4$ daily trading rules. It maintains $Q_t(a)$ for each rule, representing estimated expected daily return. Using ε -greedy, it usually picks the rule with highest $Q_t(a)$ but occasionally explores other rules. Over time, the rules that consistently yield better returns will have higher $Q_t(a)$ and will be chosen more often, while still keeping some probability of trying others in case market conditions change.

3.5 Optimistic Initial Values

One way to encourage systematic exploration without explicit randomness is to use **optimistic initialization**. Initialize:

$$Q_0(a) \text{ to a high value for all } a.$$

Because early observed rewards will usually be lower than the initial optimistic estimates, the agent is driven to try each action (arms that have not yet been tried still look very attractive). Over time, the estimates decrease toward more realistic values.

Optimistic initial values are a simple way to promote exploration in stationary problems, but with nonstationary rewards, explicit exploration (like ε -greedy) is still desirable.

3.6 Upper-Confidence-Bound (UCB) Action Selection

Upper-Confidence-Bound methods balance exploration and exploitation more systematically by using an uncertainty term. One common variant selects:

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right],$$

where $c > 0$ controls the degree of exploration.

The first term $Q_t(a)$ favors actions with high estimated value. The second term encourages actions that have been tried less often (small $N_t(a)$), reflecting higher uncertainty. As t grows, $\ln t$ grows slowly, and actions with many samples see their uncertainty term shrink.

3.7 Stationary vs Nonstationary Bandits

- **Stationary bandit:** The reward distribution for each action does not change over time. Sample averages are appropriate, and convergence to the true action values is possible if all actions are explored infinitely often.
- **Nonstationary bandit:** The reward distribution may drift over time (e.g., due to regime changes in a market). In this case, giving more weight to recent rewards is beneficial.

In nonstationary settings, using a constant step-size α :

$$Q_{t+1}(a) = Q_t(a) + \alpha(R_t - Q_t(a))$$

acts like an exponential moving average, emphasizing recent observations and allowing the agent to adapt to changes.

Example (Trading Context)

A nonstationary bandit naturally models strategies whose performance changes with market regimes. For instance, a momentum-based trading rule might perform well in trending markets but poorly in sideways markets. By using constant step-size updates, the agent can “forget” old performance and quickly adapt when the regime changes, shifting preference to strategies that currently work better.

3.8 Gradient Bandit Methods (Brief)

Gradient bandit methods directly parameterize the **preference** for each action $H_t(a)$ and convert these preferences into a probability distribution using a softmax:

$$\pi_t(a) = \frac{\exp(H_t(a))}{\sum_b \exp(H_t(b))}.$$

Preferences are updated in the direction that increases the expected reward:

$$H_{t+1}(a) = H_t(a) + \alpha(R_t - \bar{R}_t)(\mathbb{I}[A_t = a] - \pi_t(a)),$$

where $\bar{R}_t$ is a baseline (often an average reward estimate) and $\mathbb{I}$ is the indicator function. These methods optimize the policy directly, rather than estimating action values explicitly.

Additional Learning Resources

- Nothing is really required here
- But you may read Sutton and Barto chapter 2 if you want

4 Markov Decision Processes (Chapter 3)

- You’ll get many short videos on MDPs for understanding and they are enough for now.
- You may choose to watch Prof Silver’s lecture 2 on MDPs.

The multi-armed bandit formulation captures single-step decision-making, but most real-world problems, including trading, involve sequential actions where current decisions influence future situations. A **Markov Decision Process (MDP)** models such environments formally. It provides a mathematical framework for reasoning about decisions, consequences, and long-term returns.

4.1 Finite Markov Decision Processes

A finite MDP is defined by the tuple:

$$(\mathcal{S}, \mathcal{A}, P, R, \gamma),$$

where:

- $\mathcal{S}$ is a finite set of states.
- $\mathcal{A}$ is a finite set of possible actions.
- $P(s'|s, a)$ is the probability of transitioning to state s' when action a is taken in state s .
- $R(s, a, s')$ is the reward received after taking action a in state s and transitioning to s' .
- $\gamma \in [0, 1]$ is the discount factor.

MDPs express the interaction between an agent and an environment as a stochastic process:

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots$$

The objective is to find a policy that maximizes expected cumulative discounted reward.

4.2 The Markov Property

The defining assumption of MDPs is the Markov property:

$$P(S_{t+1}|S_t, A_t, S_{t-1}, A_{t-1}, \dots) = P(S_{t+1}|S_t, A_t).$$

That is, the future depends only on the present state and action, not on the history. For practical modeling, designing the state representation to be as complete as possible helps ensure the Markov assumption approximately holds.

4.3 Rewards

The reward is the immediate scalar signal of utility for the transition. For deterministic rewards:

$$R_{t+1} = R(S_t, A_t, S_{t+1}),$$

and for stochastic rewards, we define an expected value:

$$r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a].$$

Rewards emphasize local feedback, but MDP analysis is concerned with global long-term performance.

4.4 Returns and Discounting

The return is the total future reward from time t onward:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots .$$

For continuing tasks, the discounted return is used:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+1+k} .$$

The discount factor γ controls the present value of future rewards. A value close to 1 places greater importance on long-term consequences.

4.5 Policies

A policy defines how the agent behaves. A deterministic policy maps each state to one action:

$$\pi(s) = a ,$$

while a stochastic policy defines a distribution:

$$\pi(a|s) = P(A_t = a \mid S_t = s) .$$

The goal of reinforcement learning is to find an **optimal policy** π^* that maximizes expected return for all states.

4.6 State-Value Function

The state-value function under a policy π is:

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid S_t = s] .$$

It quantifies how desirable it is to start at state s when following π . Through value functions, sequential decision-making reduces to comparisons of long-term expected outcomes.

4.7 Action-Value Function

The action-value function $q_{\pi}(s, a)$ considers the immediate action:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a] .$$

It measures the expected cumulative return if action a is taken in state s and then policy π is followed. Action-values make it possible to compare actions directly rather than indirectly via future states.

4.8 Bellman Expectation Equations

The value functions satisfy recursive consistency conditions. For state-values:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v_\pi(s')].$$

This expresses that the value of a state is the expected immediate reward plus the discounted value of next states, weighted by both policy and transition probabilities.

For action-values:

$$q_\pi(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \sum_{a'} \pi(a'|s') q_\pi(s', a') \right].$$

These equations do not prescribe learning, but instead describe the mathematical target a correct estimate must satisfy.

4.9 Optimality

An optimal policy π^* achieves the maximum possible value for all states. The optimal state-value function is:

$$v^*(s) = \max_\pi v_\pi(s).$$

The Bellman optimality equation for v^* is:

$$v^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v^*(s')].$$

Similarly, optimal action-values are:

$$q^*(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a'} q^*(s', a') \right].$$

Once q^* is known, an optimal policy can be derived directly:

$$\pi^*(s) = \arg \max_a q^*(s, a).$$

This avoids explicit model reasoning at decision time.

Example (Trading Context)

In a trading MDP, the state may include factors such as recent price history, technical indicators, cash position, and holdings. Actions might be *Buy*, *Sell*, or *Hold*. Rewards represent profit or risk-adjusted returns. Once value functions are estimated, optimal decisions naturally follow: the agent selects the action whose expected long-term portfolio return is greatest, even if immediate profit is not optimal.

Additional Learning Resources

- Section 3.1
- Section 3.3's recycling robot example
- Section 3.4's cart pole balancing example
- Section 3.5, 3.6 - Gridworld and Golf examples
- Prof David Silver's lecture 2

5 Worked Example: 2×2 Gridworld (Policy Evaluation)

In this section, we explicitly compute value functions in a small Markov Decision Process. The goal is to illustrate how the Bellman expectation equations become a system of linear equations that can be solved for $v_\pi(s)$.

5.1 Environment Description

Consider a 2×2 grid with states:

$$\begin{array}{cc} S_1 & S_2 \\ S_3 & S_4 \end{array}$$

State S_4 is terminal with $v_\pi(S_4) = 0$.

At every non-terminal state (S_1, S_2, S_3):

- Actions: Up, Down, Left, Right.
- If an action hits a wall, the agent remains in the same state.
- Each action produces a constant reward of -1 .

We assume a uniform random policy:

$$\pi(a|s) = 0.25 \quad \text{for each action at non-terminal states.}$$

We set the discount factor $\gamma = 1$ (undiscounted episodic problem).

5.2 Transition Dynamics

For clarity, the transitions are deterministic:

- From S_1 : Right $\rightarrow S_2$, Down $\rightarrow S_3$, Up/Left $\rightarrow S_1$.
- From S_2 : Left $\rightarrow S_1$, Down $\rightarrow S_4$, Up/Right $\rightarrow S_2$.
- From S_3 : Up $\rightarrow S_1$, Right $\rightarrow S_4$, Down/Left $\rightarrow S_3$.
- From S_4 : Terminal.

Reward for every move (including hitting walls): $R = -1$.

5.3 Bellman Expectation Equations

Let:

$$v_1 = v_\pi(S_1), \quad v_2 = v_\pi(S_2), \quad v_3 = v_\pi(S_3), \quad v_4 = v_\pi(S_4) = 0.$$

For a non-terminal state s :

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v_\pi(s')].$$

Because $\pi(a|s) = 0.25$ and $P(s'|s, a)$ is deterministic, each action contributes $\frac{1}{4}$ of its next value.

5.4 Computing $v_\pi(S_1)$

Actions from S_1 :

- Up $\rightarrow S_1$
- Left $\rightarrow S_1$
- Right $\rightarrow S_2$
- Down $\rightarrow S_3$

Thus:

$$v_1 = 0.25(-1 + v_1) + 0.25(-1 + v_1) + 0.25(-1 + v_2) + 0.25(-1 + v_3).$$

Combine terms:

$$v_1 = 0.25(-4 + 2v_1 + v_2 + v_3) = -1 + 0.5v_1 + 0.25v_2 + 0.25v_3.$$

Rearrange:

$$v_1 - 0.5v_1 - 0.25v_2 - 0.25v_3 = -1 \implies 0.5v_1 - 0.25v_2 - 0.25v_3 = -1.$$

Multiply by 4:

$$2v_1 - v_2 - v_3 = -4. \quad (\text{Eq A})$$

5.5 Computing $v_\pi(S_2)$

Actions from S_2 :

- Up $\rightarrow S_2$
- Right $\rightarrow S_2$
- Left $\rightarrow S_1$
- Down $\rightarrow S_4$

$$v_2 = 0.25(-1 + v_2) + 0.25(-1 + v_2) + 0.25(-1 + v_1) + 0.25(-1 + 0).$$

Simplify:

$$v_2 = 0.25(-4 + 2v_2 + v_1) = -1 + 0.5v_2 + 0.25v_1.$$

Rearrange:

$$v_2 - 0.5v_2 - 0.25v_1 = -1 \implies 0.5v_2 - 0.25v_1 = -1.$$

Multiply by 4:

$$2v_2 - v_1 = -4. \quad (\text{Eq B})$$

5.6 Computing $v_\pi(S_3)$

Actions from S_3 :

- Down $\rightarrow S_3$
- Left $\rightarrow S_3$
- Up $\rightarrow S_1$
- Right $\rightarrow S_4$

$$v_3 = 0.25(-1 + v_3) + 0.25(-1 + v_3) + 0.25(-1 + v_1) + 0.25(-1 + 0).$$

$$v_3 = 0.25(-4 + 2v_3 + v_1) = -1 + 0.5v_3 + 0.25v_1.$$

Rearrange:

$$v_3 - 0.5v_3 - 0.25v_1 = -1 \implies 0.5v_3 - 0.25v_1 = -1.$$

Multiply by 4:

$$2v_3 - v_1 = -4. \quad (\text{Eq C})$$

5.7 Solving the Linear System

We have:

$$(A) : 2v_1 - v_2 - v_3 = -4$$

$$(B) : 2v_2 - v_1 = -4$$

$$(C) : 2v_3 - v_1 = -4$$

From (B):

$$2v_2 - v_1 = -4 \Rightarrow v_1 = 2v_2 + 4.$$

From (C):

$$2v_3 - v_1 = -4 \Rightarrow v_1 = 2v_3 + 4.$$

Thus:

$$2v_2 + 4 = 2v_3 + 4 \Rightarrow v_2 = v_3.$$

Let $v_2 = v_3 = x$. Then $v_1 = 2x + 4$.

Substitute into (A):

$$2(2x+4) - x - x = -4 \Rightarrow 4x+8-2x = -4 \Rightarrow 2x+8 = -4 \Rightarrow 2x = -12 \Rightarrow x = -6.$$

Therefore:

$$v_2 = -6, \quad v_3 = -6, \quad v_1 = 2(-6) + 4 = -12 + 4 = -8.$$

Since S_4 is terminal:

$$v_4 = 0.$$

5.8 Final State Values

$$v_\pi(S_1) = -8, \quad v_\pi(S_2) = -6, \quad v_\pi(S_3) = -6, \quad v_\pi(S_4) = 0.$$

These values reflect expected cumulative reward under the random policy: states closer to the terminal state tend to have higher (less negative) value.

5.9 Computing Action-Values (Optional)

For deterministic transitions and $\gamma = 1$:

$$q_\pi(s, a) = R(s, a, s') + v_\pi(s').$$

Example for S_1 :

$$q_\pi(S_1, \text{Right}) = -1 + v_\pi(S_2) = -1 + (-6) = -7,$$

$$q_\pi(S_1, \text{Down}) = -1 + v_\pi(S_3) = -1 + (-6) = -7,$$

$$q_\pi(S_1, \text{Up}) = -1 + v_\pi(S_1) = -1 + (-8) = -9,$$

$$q_\pi(S_1, \text{Left}) = -1 + v_\pi(S_1) = -9.$$

Even under a random policy, the agent can identify that actions leading toward S_2 or S_3 yield better long-term returns.

Interpretation

This example illustrates how the Bellman expectation equations encode long-term consequences. Instead of relying on intuition about distance or geometry, the value function emerges from expected cumulative reward. The solution comes from solving a coupled system of equations, a key idea behind dynamic programming.

FAQ: Common Student Doubts and Clarifications

1. Are the Bellman equations the “solution” to RL?

No. The Bellman equations *define* what the optimal values must satisfy, but they do not tell you how to compute them from experience. They describe the target, not the learning process. In model-free RL, value functions are learned from samples, not solved analytically.

2. Why do we need action-values $q(s, a)$ if we already know state-values $v(s)$?

Because the agent controls *actions*, not states. $v(s)$ tells how good a state is under a policy, but it does not tell which action to take. To choose an action using state-values, one must know model dynamics $P(s'|s, a)$ to simulate outcomes. Action-values contain this information implicitly:

$$a^* = \arg \max_a q(s, a).$$

3. Isn't it enough to “move from low value states to high value states”?

Only if the environment model is known. The agent does not know which action leads to which state. Humans see the grid or the world structure; agents experience only (s, a, r, s') samples. Without transitions, you cannot directly use $v(s)$ to choose actions.

4. Does an agent compute Bellman equations internally?

No. The agent does not solve systems of equations. It updates estimates iteratively based on experience (e.g., Q-learning or TD):

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

This process approximates the Bellman equation over time.

5. Why is Q-learning so important?

It learns optimal action-values directly from interaction, without knowing $P(s'|s, a)$ or $R(s, a, s')$. This makes it **model-free** and widely applicable to real-world problems such as trading, robotics, and control.

6. Why is reward given when leaving a state and not entering it?

The reward reflects the outcome of taking an action in a state. The pair (s, a) generates a reward and leads to a next state s' . This timing convention is

consistent across RL literature and simplifies definitions of $q(s, a)$.

7. What is the difference between Bellman expectation and optimality?

- **Expectation:** computes v_π or q_π under a given policy π .
- **Optimality:** computes v^* or q^* by maximizing over actions.

Optimality replaces the policy with $\max_a$, eliminating $\pi(a|s)$.

8. Does an optimal policy always exist?

In finite, discounted MDPs ($\gamma < 1$), at least one optimal policy exists and produces the optimal value function $v^*(s)$ for all states. Multiple optimal policies may exist, but they share the same optimal values.

9. Why doesn't the gridworld agent immediately know where each action leads?

In theoretical examples, we know $P(s'|s, a)$ and transitions for teaching purposes. In real environments, agents discover transitions through interaction. They do not see the environment graph or a transition table; they experience samples only.

10. What is the difference between model-based and model-free RL?

- **Model-based:** The agent knows or learns $P(s'|s, a)$ and R , and can plan using Bellman equations or dynamic programming.
- **Model-free:** The agent does not know the model and learns value functions or policies directly from interaction (e.g., Q-learning, TD, policy gradients).

11. If we know the full model, can we solve RL directly?

Yes. With full knowledge of $P(s'|s, a)$ and R , dynamic programming methods (value iteration, policy iteration) can solve MDPs exactly. However, this quickly becomes infeasible for large or continuous state spaces.

12. Why do we study Bellman equations if they are not practical?

They provide the theoretical backbone of RL:

- Define value functions.

- Characterize optimality.
- Justify TD and Q-learning.

Even when approximated, algorithms still aim to satisfy Bellman equations.

13. Is $q(s, a)$ always more useful than $v(s)$?

In model-free settings, $q(s, a)$ is usually more practical since it leads directly to action selection. $v(s)$ is often preferred in policy-gradient or actor-critic methods but still typically paired with q or advantage functions to aid optimization.

6 Dynamic Programming (Chapter 4)

Dynamic Programming (DP) solves reinforcement learning problems when the full environment model is known. This means the agent has access to transition probabilities $P(s'|s, a)$ and rewards $R(s, a, s')$, enabling the Bellman equations to be computed exactly. DP does not require sampling from experience; instead, value functions are updated by repeatedly evaluating consistency conditions across all states.

6.1 Policy Evaluation

Policy evaluation computes the state-value function v_π for a given policy π . The Bellman expectation equation gives:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v_\pi(s')].$$

Since this is a system of coupled equations (one per state), solving it directly is often infeasible. Instead, we apply **iterative policy evaluation**, starting from an arbitrary initial value function $v_0(s)$ and repeatedly updating:

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v_k(s')].$$

As $k \rightarrow \infty$, $v_k(s) \rightarrow v_\pi(s)$.

6.2 Policy Improvement

Once v_π is known, the policy can be improved by choosing actions that maximize expected value of the next state:

$$\pi'(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v_\pi(s')].$$

This **greedy policy** is guaranteed to be at least as good as π , or better. The improvement step chooses actions that appear best from the viewpoint of long-term expected return.

6.3 Policy Iteration

Policy iteration alternates between evaluation and improvement:

$$\pi_0 \rightarrow v_{\pi_0} \rightarrow \pi_1 \rightarrow v_{\pi_1} \rightarrow \pi_2 \rightarrow \dots$$

This sequence converges to an optimal policy π^* and its corresponding optimal value function v^* . In finite MDPs, convergence is guaranteed in a finite number of steps.

6.4 Value Iteration

Policy iteration can be slow because full policy evaluation may require many sweeps. **Value iteration** combines evaluation and improvement into a single update:

$$v_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v_k(s')].$$

This is the Bellman optimality operator. Repeated application, starting from any initial v_0 , converges to v^* . After convergence, the optimal policy follows:

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v^*(s')].$$

6.5 Asynchronous Updates

DP updates need not sweep all states simultaneously. One may update states in any order, selecting them arbitrarily or according to heuristics. This is called **asynchronous dynamic programming**. Despite the relaxed update order, convergence is preserved as long as each state continues to be updated.

6.6 Generalized Policy Iteration (GPI)

The combination of evaluation and improvement is known as **Generalized Policy Iteration**. Both processes may run in parallel or partially, gradually nudging the value function toward v^* and policies toward π^* .

Policy Evaluation $\longleftrightarrow$ Policy Improvement

Two conflicting goals:

- Evaluate the current policy accurately.
- Improve the policy toward optimal behavior.

GPI balances these forces without requiring either to complete fully before the other begins.

Example (Grid Context)

Suppose a 4×4 grid with a stochastic agent. If the full model $P(s'|s, a)$ is known, value iteration can be run directly. Starting from $v_0(s) = 0$, each iteration tightens estimates based on expected returns of neighboring states. After many sweeps, the value landscape stabilizes, and the greedy action at each cell forms an optimal path. Such solutions are widely seen in robotics navigation and shortest-path problems.

6.7 Limitations of Dynamic Programming

The primary limitation of DP is that it assumes a known model. For most real tasks:

- The agent does not know $P(s'|s, a)$ or rewards in advance.
- The state space is extremely large or continuous.
- Computing expectations over all next states is prohibitive.

Even when the model can be simulated, full DP often becomes computationally infeasible. Consequently, model-free approaches and sampled updates are more practical.

Connection to Reinforcement Learning

Dynamic Programming provides the theoretical key to optimality:

- Defines what v^* and π^* should satisfy.
- Establishes convergence principles.
- Shows how policy improvement works mathematically.

Model-free RL replaces exact expectations with sampled experience. Temporal-Difference and Q-learning approximate DP updates without access to the environment model.

7 Dynamic Programming with 2×2 Gridworld

Dynamic Programming (DP) solves reinforcement learning problems when the full model of the environment is known. Specifically, DP methods assume access to $P(s'|s, a)$ and $R(s, a, s')$, enabling the Bellman equations to be applied exactly. In this section, we illustrate the core DP ideas using a 2×2 gridworld, and demonstrate why repeated policy evaluation and improvement are required in practice.

7.1 Environment Specification

We consider the following grid:

$$\begin{array}{cc} S_1 & S_2 \\ S_3 & S_4 \end{array}$$

State S_4 is terminal. All other states allow four actions: Up, Down, Left, Right. Attempting to move into a wall results in remaining in the same state. Each move yields a reward of -1 . The discount factor is $\gamma = 1$.

We begin with a uniform random policy:

$$\pi(a|s) = 0.25 \quad \text{for all actions at nonterminal states.}$$

7.2 Policy Evaluation under the Random Policy

The Bellman expectation equation defines the value of each non-terminal state:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v_\pi(s')].$$

Solving these equations for the grid leads to:

$$v_\pi(S_1) = -8, \quad v_\pi(S_2) = -6, \quad v_\pi(S_3) = -6, \quad v_\pi(S_4) = 0.$$

These values represent the expected total number of steps (negative) until reaching the terminal state S_4 when actions are selected uniformly at random.

7.3 Policy Improvement

Once v_π is known, we construct a new policy π' by selecting greedy actions with respect to v_π :

$$\pi'(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v_\pi(s')].$$

For example, at S_1 :

$$q_\pi(S_1, \text{Right}) = -1 + v_\pi(S_2) = -7,$$

$$q_\pi(S_1, \text{Down}) = -1 + v_\pi(S_3) = -7,$$

while actions that remain in S_1 have value -9 . Therefore:

$$\pi'(S_1) \in \{\text{Right}, \text{Down}\}.$$

Conducting this improvement at all states yields a strictly better policy. By the Policy Improvement Theorem, π' satisfies:

$$v_{\pi'}(s) \geq v_\pi(s) \quad \forall s.$$

7.4 Why One Round is Not Enough

Evaluating v_π captures the performance of the *current* (possibly poor) policy. A single improvement step only produces a locally better strategy. The updated policy will itself lead to new state visitation patterns and new long-term returns. These must be re-evaluated to obtain the correct value function for the improved policy.

Thus, optimality requires repeated alternation of:

$$(\text{evaluate policy}) \rightarrow (\text{improve policy}).$$

A policy may appear greedy with respect to v_π but still fail to be optimal if v_π itself comes from a suboptimal policy.

7.5 Policy Iteration

Policy iteration repeats evaluation and improvement:

$$\pi_0 \rightarrow v_{\pi_0} \rightarrow \pi_1 \rightarrow v_{\pi_1} \rightarrow \dots$$

The process terminates when the policy stabilizes:

$$\pi_{k+1} = \pi_k.$$

At this point, π_k and v_{π_k} are optimal for finite MDPs.

In the 2×2 grid, starting from the random policy, evaluation yields v_π , which then improves the policy towards selecting actions that move directly toward S_4 . Re-evaluating this new policy results in less negative values until convergence.

7.6 Value Iteration

Policy iteration may require many sweeps for full evaluation. **Value iteration** avoids full evaluation by applying the Bellman optimality update directly:

$$v_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v_k(s')].$$

This update simultaneously improves the value function and implicitly improves the policy. After convergence, the optimal policy is:

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma v^*(s')].$$

In the 2×2 grid, starting from $v_0(s) = 0$, repeated Bellman updates quickly converge to the same values found using policy iteration.

7.7 Generalized Policy Iteration

Dynamic programming can be seen as two interacting processes:

$$\text{Policy evaluation: } \pi \rightarrow v_\pi, \quad \text{Policy improvement: } v \rightarrow \pi'.$$

Each operation drives the other: value functions adjust to current policies, and policies adjust to current value functions. This interaction is called Generalized Policy Iteration (GPI). Convergence emerges even when neither process is completed before the other begins.

7.8 Scaling Beyond the 2×2 Example

For larger MDPs, the technique is identical:

1. Evaluate the current policy.
2. Improve the policy using greedy actions.
3. Repeat until policy stabilizes.

However, values and transitions become too numerous to solve by hand. Policy iteration or value iteration must be performed computationally, often approximated or made asynchronous. For complex environments such as trading, robotics, or high-dimensional games, this DP process becomes infeasible, leading naturally to model-free methods.

Key Insight: *Dynamic programming provides exact solutions only when the full model is known. The alternation of evaluation and improvement is essential to reach optimal policies, even in small examples.*

8 Monte Carlo Methods (Chapter 5)

Monte Carlo (MC) methods learn from **complete episodes of experience**. They require no knowledge of the environment dynamics $P(s'|s, a)$ and no state transition model. Instead, value functions are learned by averaging sampled returns obtained through interaction with the environment.

Monte Carlo methods are therefore a cornerstone of **model-free reinforcement learning**.

8.1 Key Idea

Instead of computing expectations analytically, Monte Carlo approximates them via sampling:

$$v_\pi(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)},$$

where $G_t^{(i)}$ is the return observed when visiting state s at time t in the i -th episode.

This works because of the **Law of Large Numbers**: empirical averages converge to their true expectations when sampled repeatedly.

8.2 First-Visit and Every-Visit Monte Carlo

Monte Carlo value estimates can be computed in two ways.

- **First-Visit MC**: Only the first time a state is visited per episode contributes a training sample.
- **Every-Visit MC**: Every occurrence of the state in the episode is used as a sample.

Both methods converge to v_π as episode count increases, but they may differ in variance.

8.3 Monte Carlo for Action Values

For control problems, we may not know the model, so $q_\pi(s, a)$ must be estimated directly:

$$q_\pi(s, a) \approx \frac{1}{N(s, a)} \sum_{i=1}^{N(s, a)} G_t^{(i)},$$

where returns are collected from episodes when action a was taken in state s .

This removes the need to sum over $P(s'|s, a)$ and makes Monte Carlo suitable for large unknown environments.

8.4 Exploration Requirement

A policy must visit state–action pairs sufficiently often in order to learn their values. For control, this is critical. Pure greedy policies tend to become trapped in suboptimal behavior.

To guarantee coverage, we use **exploring starts** or **stochastic soft policies**.

8.4.1 Exploring Starts

Each episode begins from a randomly selected (s, a) pair. While theoretically useful, exploring starts are rarely practical in large or real environments.

8.4.2 ϵ -Soft Policies

We maintain a policy π such that:

$$\pi(a|s) \geq \frac{\epsilon}{|\mathcal{A}(s)|},$$

ensuring each action has some nonzero selection probability.

This allows state–action values to be sampled indefinitely, enabling convergence.

8.5 Monte Carlo Control with ε -Soft Policies

Monte Carlo control alternates between estimation and improvement:

$$\text{estimate } q_\pi \rightarrow \text{improve } \pi.$$

Policy improvement uses greedy-with-respect-to- q :

$$\pi'(s) = \arg \max_a q_\pi(s, a),$$

combined with ε exploration:

$$\pi(a|s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{if } a = \arg \max_a q(s, a), \\ \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{otherwise.} \end{cases}$$

This approach is proven to converge to an optimal policy π^* .

8.6 Monte Carlo vs Dynamic Programming

Monte Carlo does not require a transition model. Instead of computing expectations explicitly as in DP, it estimates them by sampling:

$$\sum_{s', a'} P(s'|s, a)[\cdot] \quad \text{replaced by empirical returns } G.$$

Monte Carlo remains effective even when the model is unknown or intractable.

Example (Trading Context)

An RL trading agent interacts with market data daily. Each trading week forms an episode. Returns accumulated across the week form the return G_t for each visited state. Monte Carlo prediction estimates $v_\pi(s)$ by averaging weekly outcomes. For control, the strategy chooses buy/sell/hold actions based on $q_\pi(s, a)$ while keeping some exploration to avoid premature convergence to suboptimal strategies.

8.7 Limitations of Monte Carlo Methods

- Episodes must terminate for returns to be computed.
- High variance due to full-return estimates.
- Convergence may be slow in large state spaces.

These limitations lead naturally to **Temporal Difference (TD)** methods, which bootstrap estimates without waiting for episode termination.

9 Incremental Monte Carlo Methods

Traditional Monte Carlo methods estimate value functions by averaging all sampled returns. Given a policy π , and observed returns $G^{(1)}, G^{(2)}, \dots, G^{(N)}$ for a state s , the first-visit method computes:

$$v_\pi(s) = \frac{1}{N} \sum_{i=1}^N G^{(i)}.$$

While correct, this requires storing all past returns or maintaining visit counts. In large or continuous environments this is impractical.

9.1 Incremental Monte Carlo Update

To avoid storing previous returns, value estimates are updated incrementally using a step-size:

$$v(s) \leftarrow v(s) + \alpha(G - v(s)),$$

where G is the return observed from the most recent episode, and $\alpha \in (0, 1]$ is a learning rate.

This update shifts the estimate toward the new observed return, without recomputation of historical averages.

9.2 Connection to Sample Average

Choosing

$$\alpha = \frac{1}{N(s)},$$

where $N(s)$ is the visit count, recovers the exact sample average solution:

$$v_N(s) = v_{N-1}(s) + \frac{1}{N(s)}(G - v_{N-1}(s)).$$

Thus, the incremental formula is an algebraic reformulation of averaging.

9.3 Advantages of Constant Step-Size

A constant α does not produce exact sample averages, but has key advantages:

- Suitable for nonstationary environments where the true value may change over time.
- Gives greater weight to recent experiences (exponential moving average).
- Enables online learning without storing history.

The update is then:

$$v(s) \leftarrow (1 - \alpha)v(s) + \alpha G,$$

which smoothly integrates new observations.

9.4 Incremental Monte Carlo for Action Values

Control requires values of state–action pairs. Incremental Monte Carlo for $q(s, a)$ is analogous:

$$q(s, a) \leftarrow q(s, a) + \alpha(G - q(s, a)).$$

Only the observed return G and current estimate $q(s, a)$ are required.

Example (Trading Context)

Suppose a trading policy is evaluated at the end of each week. The weekly profit is used as G , the return. Instead of storing profits from every past week, the estimate is updated online:

$$q(\text{state}, \text{Buy}) \leftarrow q(\text{state}, \text{Buy}) + \alpha(G - q(\text{state}, \text{Buy})).$$

If market conditions change, recent performance naturally becomes dominant in the value estimate.

9.5 Summary

Incremental Monte Carlo:

- Avoids storing full return histories.
- Supports continual learning.
- Handles nonstationary environments.
- Can be applied to $v(s)$ or $q(s, a)$.

It is particularly useful in large-scale or streaming applications where full averaging is infeasible.

10 Temporal Difference Learning (Chapter 6)

Temporal Difference (TD) methods estimate value functions by bootstrapping: they update estimates based on other learned estimates rather than waiting for full episode returns. This makes TD learning a bridge between Monte Carlo (sampled returns) and Dynamic Programming (bootstrapped expectations).

TD learning is **model-free** and works from raw experience, but unlike Monte Carlo, it does not require complete episodes to compute targets.

10.1 TD Prediction

The goal is to approximate $v_\pi(s)$ for a given policy π , without knowing the model $P(s'|s, a)$ or waiting until episode termination.

TD(0) uses the one-step update:

$$v(S_t) \leftarrow v(S_t) + \alpha(R_{t+1} + \gamma v(S_{t+1}) - v(S_t)),$$

where α is the learning rate.

The quantity

$$\delta_t = R_{t+1} + \gamma v(S_{t+1}) - v(S_t)$$

is the **TD error**. It measures how much the new experience disagrees with the current estimate. The update moves $v(S_t)$ in the direction of smaller TD error.

10.2 Monte Carlo vs TD Learning

Monte Carlo methods compute returns using complete episodes:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

and update:

$$v(S_t) \leftarrow v(S_t) + \alpha(G_t - v(S_t)).$$

TD(0) uses bootstrapping from the next state value:

$$v(S_t) \leftarrow v(S_t) + \alpha(R_{t+1} + \gamma v(S_{t+1}) - v(S_t)).$$

The difference:

- Monte Carlo waits until termination, uses actual returns.
- TD updates online, uses predicted value of *next* state.

10.3 How TD Predicts the Value of the Next State

Temporal Difference (TD) learning does not require full episode returns. Instead, it uses the current estimate of the value of the next state to update the current one. This mechanism is called **bootstrapping**.

TD(0) uses the update:

$$v(S_t) \leftarrow v(S_t) + \alpha(R_{t+1} + \gamma v(S_{t+1}) - v(S_t)),$$

where the quantity

$$R_{t+1} + \gamma v(S_{t+1})$$

is the **TD target**. It is a one-step estimate of future return based on the reward just observed and the current estimate of the next state.

Interpretation

- TD does not know the true value of S_{t+1} .
- It uses *its own current estimate* $v(S_{t+1})$.
- This estimate becomes more accurate as learning proceeds.
- The agent continually corrects its predictions using experience.

Instead of waiting until the end of an episode (Monte Carlo), TD updates values at every step.

Example

Suppose at time t the agent is in state S_t and receives a reward $R_{t+1} = +5$ after moving to S_{t+1} . Let:

$$v(S_t) = 10, \quad v(S_{t+1}) = 20, \quad \gamma = 0.9, \quad \alpha = 0.1.$$

The TD target is:

$$T = R_{t+1} + \gamma v(S_{t+1}) = 5 + 0.9 \cdot 20 = 23.$$

The TD error is:

$$\delta = T - v(S_t) = 23 - 10 = 13.$$

The update becomes:

$$v(S_t) \leftarrow v(S_t) + \alpha \delta = 10 + 0.1 \cdot 13 = 11.3.$$

The estimate of $v(S_t)$ moves slightly toward the new target. No episode termination is required.

Key Insight

- TD learning predicts the value of the next state using its own current estimates.
- These predictions gradually improve as more experience accumulates.
- This makes TD suitable for online, streaming, and continuous tasks.

10.4 TD Advantages

- Does not require complete episodes.
- Learn online, step-by-step.
- Lower variance than Monte Carlo.
- Suitable for continuing tasks.

10.5 TD and the Markov Property

TD(0) is correct when the Markov assumption holds, because S_{t+1} contains all necessary information to estimate future returns. The next-state bootstrap is unbiased in expectation under the true value v_π .

10.6 TD for Continuing Tasks

For tasks without episode termination, discounted returns are standard. TD methods can learn continually, updating each step. Monte Carlo cannot operate without artificial termination or time windows.

10.7 Batch Monte Carlo vs Batch TD

When trained on a fixed set of episodes:

- **Monte Carlo** finds estimates that minimize squared error to final returns.
- **TD** finds estimates consistent with the Bellman equation.

Thus, TD is “dynamic-programming consistent,” while Monte Carlo is “return consistent.” This difference matters when dealing with incomplete knowledge or sparse sampling.

10.8 The TD Prediction Target

Monte Carlo target:

$$G_t$$

TD(0) target:

$$R_{t+1} + \gamma v(S_{t+1})$$

This target is a one-step approximation of v_π and is refined as v becomes accurate.

10.9 Eligibility Traces (Brief)

Longer TD horizons interpolate between TD(0) and Monte Carlo. TD(λ) blends n -step returns using a decay factor λ . This provides a continuum between no bootstrapping (Monte Carlo) and full bootstrapping (TD(0)).

Example (Trading Context)

In a trading environment, market data arrives continuously. Monte Carlo requires complete episodes (e.g., monthly or yearly summaries) to update. TD learning allows updating value estimates at every timestep:

$$v(\text{current market state}) \leftarrow v + \alpha(\text{daily reward} + \gamma v(\text{next state}) - v),$$

which makes it suitable for real-world streaming markets.

10.10 Summary

Temporal Difference methods:

- Learn from raw experience.
- Do not require transition models.
- Update online without full returns.
- Bootstrap using learned estimates.

They unify Monte Carlo sampling and Dynamic Programming bootstrapping, and form the foundation of effective model-free RL.

11 Temporal Difference Control: SARSA and Q-learning

Temporal Difference (TD) control extends TD prediction to derive optimal policies. Instead of only evaluating a fixed policy, TD control improves the policy based on value estimates, eventually converging to π^* and q^* .

TD control methods rely on the state-action value function $q(s, a)$, because the optimal policy depends on selecting the best actions rather than only evaluating states.

11.1 On-Policy vs Off-Policy Control

Control algorithms differ in how they treat the behavior policy:

- **On-policy** methods evaluate and improve the same policy that generates data.
- **Off-policy** methods evaluate the greedy target policy while acting with a different behavior policy.

This distinction leads to two core TD algorithms:

1. SARSA (on-policy)
2. Q-learning (off-policy)

11.2 SARSA: On-Policy TD Control

SARSA learns the value of the policy it actually follows. The update depends on the action *chosen by the policy*, not the greedy optimal action.

For a transition $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right).$$

The update target is:

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}),$$

which is the value of the next state–action pair under the current policy.

Exploration and ε -soft Policies

Since the policy improves over time, SARSA must continue exploring. A typical implementation uses ε -greedy:

$$\pi(a|s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|}, & a = \arg \max_a Q(s, a), \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & \text{otherwise.} \end{cases}$$

SARSA converges to an optimal policy when ε decreases appropriately and all state–action pairs are explored infinitely often.

11.3 Q-Learning: Off-Policy TD Control

Q-learning learns the value of the *optimal* policy independent of how data is generated. The learning policy may explore, but the update target assumes greedy behavior.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right).$$

Here, the target policy is greedy:

$$\max_{a'} Q(S_{t+1}, a'),$$

even if the behavior policy is exploratory.

11.4 Clarifications on Q-learning: Common Questions

1. Why does Q-learning use $\max_{a'} Q(s', a')$ instead of $Q(s', a_{\text{actual}})$?

Q-learning is **off-policy**. It learns the value of the *optimal future behavior*, regardless of the current exploratory behavior. The update target is:

$$R_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'),$$

which assumes that from the next state onward, the agent behaves optimally. This separates exploration (behavior policy) from learning (target policy).

2. Can Q-learning increase the value of a bad action?

Yes. If the immediate reward is negative but the next state has high potential under the optimal action, then:

$$R_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$$

may still be positive. Q-learning is optimistic: it assumes the agent will recover from mistakes by acting optimally in the future. This is a direct consequence of the Bellman optimality equation.

3. How does Q-learning update the policy?

Q-learning updates the policy *indirectly* via changes in the Q-values. The greedy policy is:

$$\pi(s) = \arg \max_a Q(s, a).$$

No explicit policy update step is required. As state-action values evolve, the greedy action in each state changes, which yields policy improvement.

4. Why do we still use ε -greedy during training?

Exploration is required so that all state-action pairs receive sufficient updates. The behavior policy is usually:

$$\varepsilon\text{-greedy over } Q(s, a),$$

so that with probability $1 - \varepsilon$ the best action is chosen, and with probability ε a random action is selected. This ensures learning does not prematurely converge to suboptimal actions.

5. Does Q-learning care about the next action chosen by the agent?

No. Q-learning ignores the actual next action and uses the best possible action:

$$\max_{a'} Q(s', a').$$

This is why it can learn an optimal policy *even if the agent explores randomly* during training.

6. Why is Q-learning considered “optimistic”?

Because improvements are driven by the **best possible future**, not the observed future. The target assumes:

optimal behavior from s' onward, even if behavior is not optimal now.

This makes Q-learning robust to occasional mistakes, but can cause overestimation in noisy environments.

Summary

- Q-learning learns the *optimal policy* while behaving with an exploratory policy.
- It updates toward the best possible future return, not the actual chosen next action.
- Policy improvement emerges from Q-values via greedy selection.
- Temporary bad actions may still be valued positively if recovery potential is high.

Practical Difference

- SARSA: Learns safety-conscious behavior (e.g., avoids risky high-variance actions).
- Q-learning: Learns aggressively optimal behavior (e.g., assumes optimal future decisions).

This difference is critical in noisy or stochastic environments.

11.5 Example Context (Trading)

In trading:

- SARSA: learns conservative execution strategies that account for volatility.
- Q-learning: assumes future decisions will be optimal and may adopt riskier positions.

Both can work, but Q-learning tends to be more brittle without stabilizing techniques.

11.6 Convergence Conditions

For tabular Q-learning and SARSA to converge:

1. All state-action pairs are visited infinitely often.
2. Learning rate decays over time ($\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$).
3. Policy becomes greedy in the limit.

These conditions are rarely met in large, continuous environments, motivating function approximation.

11.7 Expected SARSA

Expected SARSA replaces the bootstrap target with an expectation under the policy:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \sum_{a'} \pi(a'|S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t) \right).$$

This reduces variance, especially under stochastic policies.

11.8 Summary

- SARSA is on-policy: evaluates what it actually does.
- Q-learning is off-policy: learns greedy value while exploring.
- Both bootstrap from $Q(s, a)$ and do not need environment models.
- These algorithms form the foundation of practical, model-free RL control.

TD control completes the bridge from policy evaluation to policy improvement, enabling autonomous agents to learn from experience alone.

12 Function Approximation (Chapter 9)

In large or continuous state spaces, tabular methods become infeasible. Instead of storing a value for each state or state–action pair, we approximate the value function using a parameterized function. This idea is central to real-world reinforcement learning.

12.1 Motivation

Tabular methods require:

$$\mathcal{O}(|\mathcal{S}| \times |\mathcal{A}|),$$

which is practical only when both the state and action spaces are small.

In domains such as trading, robotics, or autonomous driving:

- State is continuous or high-dimensional.
- Many states are never visited.
- Generalization is required to learn efficiently.

Function approximation replaces lookup tables with learned models.

12.2 Value Function Approximation

We define a parameterized approximation:

$$\hat{v}(s; \theta) \approx v_\pi(s),$$

where θ is a vector of weights.

The goal becomes:

$$\theta = \arg \min_{\theta} \mathbb{E}[(v_\pi(s) - \hat{v}(s; \theta))^2].$$

Parameters θ are adjusted from sampled experience.

12.3 Linear Function Approximation

The simplest model is linear:

$$\hat{v}(s; \theta) = \theta^\top x(s),$$

where $x(s)$ is a feature vector.

For TD(0), the update rule becomes:

$$\theta \leftarrow \theta + \alpha \left(R_{t+1} + \gamma \hat{v}(S_{t+1}; \theta) - \hat{v}(S_t; \theta) \right) \nabla_{\theta} \hat{v}(S_t; \theta).$$

With linear models:

$$\nabla_{\theta} \hat{v}(s; \theta) = x(s).$$

Thus:

$$\theta \leftarrow \theta + \alpha \delta_t x(S_t),$$

where

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; \theta) - \hat{v}(S_t; \theta).$$

12.4 Action-Value Approximation

We approximate $q_\pi(s, a)$ directly:

$$\hat{q}(s, a; \theta) \approx q_\pi(s, a).$$

Updates follow the same TD rule:

$$\theta \leftarrow \theta + \alpha \left(R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}; \theta) - \hat{q}(S_t, A_t; \theta) \right) \nabla_{\theta} \hat{q}(S_t, A_t; \theta).$$

This enables TD control methods such as SARSA and Q-learning to scale to complex tasks.

12.5 Generalization

Function approximation introduces generalization:

- Unseen states can inherit useful structure.
- Updates in one state influence neighboring regions of state space.
- Learning becomes sample-efficient.

However, incorrect generalization creates instability if not handled carefully.

12.6 Nonlinear Approximation: Neural Networks

Nonlinear models such as multilayer perceptrons represent complex value functions:

$$\hat{q}(s, a; \theta) = \text{NN}_\theta(s, a).$$

Training proceeds by stochastic gradient descent:

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_\theta \hat{q}(S_t, A_t; \theta),$$

where δ_t is the TD error.

Neural approximators enable high-dimensional RL, but introduce challenges:

- Nonstationary targets
- Correlated input samples
- Divergence due to bootstrapping

These issues motivate later techniques such as experience replay and target networks (DQN).

12.7 Approximation and Control

When function approximation interacts with control, learning becomes unstable:

- bootstrapping (TD)
- off-policy updates (Q-learning)
- nonlinear approximation (neural nets)

This combination is known as the **deadly triad**. Practical deep RL systems use stabilizing mechanisms (target networks, replay buffers).

Example (Trading Context)

In a trading system, state s may contain:

- Historical candlesticks
- Moving averages
- RSI or MACD
- Market volatility

These are continuous signals. A tabular Q-table cannot enumerate all states. A neural network approximator:

$$\hat{q}(s, a; \theta)$$

generalizes across similar patterns and learns profitable policies.

12.8 Summary

Function approximation:

- Replaces lookup tables with parametric models.
- Enables RL in continuous and high-dimensional environments.
- Introduces generalization and sample efficiency.
- Requires careful design to avoid instability.

It is the foundation of modern deep reinforcement learning.

12.9 Example: Linear Function Approximation with Q-learning (Step-by-Step)

This example illustrates how a parameterized value function replaces a Q-table in continuous or high-dimensional state spaces. We use a linear approximator for the action-value function:

$$\hat{Q}(s, a; \theta_a) = \theta_a^\top x(s),$$

where $x(s)$ is a feature vector and θ_a is a weight vector for action a .

State, Features, and Initial Parameters

Let the feature vector for the current state s be:

$$x(s) = \begin{bmatrix} 1.000 \\ 0.500 \\ -0.200 \end{bmatrix}.$$

Assume two actions: Buy and Sell, each with its own parameter vector:

$$\theta_{\text{Buy}} = \begin{bmatrix} 0.200 \\ -0.100 \\ 0.050 \end{bmatrix}, \quad \theta_{\text{Sell}} = \begin{bmatrix} 0.000 \\ 0.100 \\ -0.020 \end{bmatrix}.$$

Step 1: Evaluate Current Q-values

Using $\hat{Q}(s, a) = \theta_a^\top x(s)$:

$$\hat{Q}(s, \text{Buy}) = 0.200 - 0.050 - 0.010 = 0.140,$$

$$\hat{Q}(s, \text{Sell}) = 0.050 + 0.004 = 0.054.$$

The agent chooses action Buy.

Step 2: Observe Transition

Immediate reward:

$$r = -0.50,$$

discount factor:

$$\gamma = 0.9.$$

Next-state features:

$$x(s') = \begin{bmatrix} 1.000 \\ 0.400 \\ 0.100 \end{bmatrix}.$$

Evaluate next-state Q-values:

$$\hat{Q}(s', \text{Buy}) = 0.200 - 0.040 + 0.005 = 0.165,$$

$$\hat{Q}(s', \text{Sell}) = 0.040 - 0.002 = 0.038.$$

Thus:

$$\max_{a'} \hat{Q}(s', a') = 0.165.$$

Step 3: Compute TD Target and TD Error

Q-learning target:

$$\text{target} = r + \gamma \max_{a'} \hat{Q}(s', a') = -0.50 + 0.9 \cdot 0.165 = -0.3515.$$

TD error:

$$\delta = \text{target} - \hat{Q}(s, \text{Buy}) = -0.3515 - 0.140 = -0.4915.$$

Step 4: Gradient of the Linear Approximator

Since:

$$\hat{Q}(s, a; \theta_a) = \theta_a^\top x(s),$$

its gradient is simply:

$$\nabla_{\theta_a} \hat{Q}(s, a; \theta_a) = x(s).$$

Step 5: Parameter Update (Q-learning with Gradient Descent)

Using learning rate $\alpha = 0.10$:

$$\Delta\theta_{\text{Buy}} = \alpha \delta x(s) = 0.10 \times (-0.4915) \times \begin{bmatrix} 1 \\ 0.5 \\ -0.2 \end{bmatrix} = \begin{bmatrix} -0.04915 \\ -0.024575 \\ 0.00983 \end{bmatrix}.$$

Updated parameters:

$$\theta_{\text{Buy,new}} = \begin{bmatrix} 0.200 \\ -0.100 \\ 0.050 \end{bmatrix} + \begin{bmatrix} -0.04915 \\ -0.024575 \\ 0.00983 \end{bmatrix} = \begin{bmatrix} 0.15085 \\ -0.124575 \\ 0.05983 \end{bmatrix}.$$

Step 6: Effect on the Q-function

Recompute:

$$\hat{Q}_{\text{new}}(s, \text{Buy}) = 0.15085 - 0.124575 \cdot 0.5 + 0.05983 \cdot (-0.2) \approx 0.0766.$$

Thus, the Q-value for Buy at state s decreased from 0.140 to approximately 0.0766, reflecting the poor observed outcome.

Key Insights

- Function approximation replaces table entries with a smooth function over features.
- The parameters θ generalize updates to many states with similar features.
- Linear models have simple gradients: $\nabla_{\theta} \hat{Q} = x(s)$.
- The Q-learning target uses $\max_{a'} \hat{Q}(s', a')$, implementing off-policy control.
- Parameter updates move the function toward the Bellman optimality solution.

13 Deep Q-Networks (DQN)

Deep Q-Networks (DQN) extend Q-learning by replacing the Q-table with a neural network $\hat{Q}(s, a; \theta)$. This enables Q-learning to operate in large and continuous state spaces where tabular representations are impossible. DQN was first introduced by DeepMind and famously achieved human-level performance on Atari games directly from raw pixels.

This chapter introduces the core components of DQN:

- neural network function approximation for $Q(s, a)$,
- experience replay,

- target networks,
- stable mini-batch TD learning.

13.1 Motivation

Tabular Q-learning fails in large or continuous state spaces:

$Q(s, a)$ cannot be stored as a table when s is high-dimensional.

Neural networks provide generalization:

$$\hat{Q}(s, a; \theta) \approx Q(s, a),$$

mapping states to action-values through learned parameters θ .

However, non-linear approximators with bootstrapping and off-policy learning cause instability (the “deadly triad”). DQN introduces two mechanisms to stabilize learning:

1. a **replay buffer**,
2. a **target network**.

13.2 Q-learning with a Neural Network

For a transition (s, a, r, s') , the DQN target is:

$$y = r + \gamma \max_{a'} \hat{Q}(s', a'; \theta^-),$$

where θ^- are the parameters of the target network.

The loss function for the online network with parameters θ is:

$$L(\theta) = \left(y - \hat{Q}(s, a; \theta) \right)^2.$$

Updates occur by gradient descent:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta).$$

13.3 Experience Replay

DQN stores transitions in a buffer $\mathcal{D}$:

$$\mathcal{D} = \{(s_t, a_t, r_{t+1}, s_{t+1})\}.$$

At each learning step, a mini-batch is sampled uniformly from $\mathcal{D}$. This improves stability by:

- breaking correlation between sequential samples,

- smoothing the data distribution,
- enabling efficient reuse of past experience.

Mini-batch target for example i :

$$y_i = r_i + \gamma \max_{a'} \hat{Q}(s'_i, a'; \theta^-),$$

and loss:

$$L = \frac{1}{N} \sum_{i=1}^N \left(y_i - \hat{Q}(s_i, a_i; \theta) \right)^2.$$

13.4 Target Network

A separate target network $\hat{Q}(s, a; \theta^-)$ is maintained to stabilize targets. Its parameters are periodically copied:

$$\theta^- \leftarrow \theta \quad \text{every } C \text{ steps.}$$

Benefits:

- reduces oscillations,
- prevents chasing a moving target,
- improves convergence.

13.5 DQN Algorithm

Initialize:

- Replay buffer $\mathcal{D}$
- Online Q-network with weights θ
- Target network with weights $\theta^- = \theta$

Loop over episodes:

1. Observe state s
2. Select action a using ε -greedy over $\hat{Q}(s, a; \theta)$
3. Execute a , observe r and next state s'
4. Store (s, a, r, s') in replay buffer $\mathcal{D}$
5. Sample random mini-batch $\{(s_i, a_i, r_i, s'_i)\}$ from $\mathcal{D}$
6. Compute targets:

$$y_i = r_i + \gamma \max_{a'} \hat{Q}(s'_i, a'; \theta^-)$$

7. Update online network by minimizing:

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{Q}(s_i, a_i; \theta))^2$$

8. Every C steps, update target network:

$$\theta^- \leftarrow \theta$$

13.6 Why DQN Works

DQN stabilizes value-based deep RL by:

- interrupting correlations in data (replay buffer),
- slowing the movement of targets (target network),
- using mini-batch updates (reducing variance),
- enabling neural networks to predict high-dimensional Q-values.

13.7 Limitations of DQN

- Supports only **discrete actions**.
- Overestimates Q-values due to the max operator.
- Sensitive to hyperparameters.
- Training can still be unstable in noisy environments (e.g., financial markets).

These limitations motivate improved algorithms such as:

- Double DQN (reduces overestimation)
- Dueling DQN (better state representation)
- Prioritized Replay (sample important transitions more often)

13.8 Relevance to Trading

In trading environments:

- State is high-dimensional (prices, indicators, volatility).
- Actions are discrete (Buy/Sell/Hold).
- Rewards may be sparse or noisy.

DQN provides a foundation for learning policies from raw market data, but care must be taken to design:

- proper normalization of input features,
- realistic reward functions,
- transaction cost modeling,
- training on decorrelated samples.

13.9 Summary

- DQN is Q-learning with neural networks.
- Experience replay and target networks ensure stability.
- Mini-batch TD updates reduce variance.
- Supports large, continuous state spaces with discrete action sets.

DQN marks the transition from classical RL to practical deep RL systems.

14 Improvements to DQN: Double DQN, Dueling DQN, and Prioritized Replay

Although Deep Q-Networks (DQN) represented a major breakthrough, they still exhibit notable weaknesses:

- systematic overestimation of Q-values,
- inefficient learning from uniformly sampled replay buffers,
- poor state representation when some states do not require choosing actions.

This chapter introduces three major extensions that significantly improve stability, accuracy, and sample efficiency.

14.1 Double DQN

Standard DQN uses the same network both to select the best next action and to evaluate it:

$$y = r + \gamma \max_{a'} \hat{Q}(s', a'; \theta^-).$$

Due to noise, this leads to *overestimation*:

$$\mathbb{E}[\max_a X_a] \geq \max_a \mathbb{E}[X_a].$$

Double DQN decouples action selection from action evaluation.

Double DQN Target

1. Use the online network (parameters θ) to select the next action:

$$a = \arg \max_{a'} \hat{Q}(s', a'; \theta).$$

2. Use the target network (parameters θ^-) to evaluate it:

$$y = r + \gamma \hat{Q}(s', a'; \theta^-).$$

This prevents upward bias by separating selection and evaluation.

Benefits

- strong reduction in Q-value overestimation,
- more accurate value estimates,
- better and more stable learning.

14.2 Dueling DQN

In many states, the choice of action does not significantly impact value. For example:

- terminal states,
- states with redundant actions,
- states where the value is dominated by state quality rather than action.

Standard DQN must learn $Q(s, a)$ separately for each action, even when unnecessary. Dueling DQN introduces two separate streams inside the network:

$$V(s) \quad \text{and} \quad A(s, a),$$

representing:

- state-value function $V(s)$,
- advantage function $A(s, a) = Q(s, a) - V(s)$.

Dueling Aggregation

The Q-value is recombined as:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right).$$

This architecture helps the network learn state-values even when action choices do not matter.

Benefits

- faster learning of state importance,
- improved policy evaluation,
- better performance in environments with irrelevant actions.

14.3 Prioritized Experience Replay

Uniform replay samples transitions at random, treating all experiences equally. However, some transitions are more informative—those with higher TD error.

Prioritized replay samples transitions with probability proportional to:

$$P(i) = \frac{|\delta_i|^\alpha}{\sum_j |\delta_j|^\alpha},$$

where:

$$\delta_i = y_i - \hat{Q}(s_i, a_i; \theta)$$

is the TD error.

Bias Correction

To avoid introducing sampling bias, importance-sampling weights are used:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta.$$

These weights downscale updates for highly probable samples.

Benefits

- focuses learning on transitions with high TD error,
- dramatically speeds up learning,
- improves sample efficiency.

14.4 Summary of Improvements

Double DQN

- Fixes Q-value overestimation.
- Uses separate networks for action selection and evaluation.

Dueling DQN

- Separates state-value and advantage.
- Learns better representations for states.

Prioritized Replay

- Samples important transitions more often.
- Uses TD error as a priority signal.

14.5 Relevance to Trading

These improvements are especially relevant in financial environments:

- **Double DQN** avoids overestimating profitable trades.
- **Dueling networks** help understand market states where action choice (Buy/Hold/Sell) is less impactful.
- **Prioritized replay** accelerates learning from rare but important market transitions (breakouts, reversals, large drawdowns).

14.6 Conclusion

DQN becomes far more stable and sample-efficient when augmented with:

- Double Q-learning,
- Dueling architecture,
- Prioritized replay.

These techniques form the basis of modern value-based deep RL systems, and are commonly used before transitioning to policy gradient and actor–critic methods.

15 Policy Gradient Methods (REINFORCE)

Value-based methods like Q-learning choose actions by estimating action-values and acting greedily. Policy gradient methods directly learn a parameterized policy:

$$\pi(a|s; \theta)$$

which outputs a probability distribution over actions for each state. They optimize performance by ascending the gradient of expected return.

Policy gradients are the foundation of modern algorithms such as REINFORCE, Actor–Critic, PPO, SAC, and TRPO.

15.1 Why Policy Gradients?

Value-based methods struggle with:

- large or continuous action spaces,
- stochastic policies,
- smooth control requirements,
- unstable greedy updates.

Policy gradients solve these issues by:

- directly optimizing the policy,
- producing smooth probability distributions,
- using differentiable parameterizations (neural networks),
- naturally fitting continuous action spaces.

15.2 Objective Function

We define the performance measure of a policy as:

$$J(\theta) = \mathbb{E}_{\pi}[G_t],$$

where G_t is the return following the policy $\pi(\cdot|\cdot; \theta)$.

The goal is to adjust θ to maximize $J(\theta)$.

15.3 Policy Gradient Theorem

The theorem states:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi(A_t|S_t; \theta) Q_{\pi}(S_t, A_t)].$$

This remarkable result means:

- we do not need to differentiate through the environment,
- the gradient depends only on $\log \pi$ and Q_{π} (or an estimate),
- stochastic policies are naturally supported.

15.4 REINFORCE: Monte Carlo Policy Gradient

REINFORCE uses the return G_t as an unbiased sample of $Q_{\pi}(S_t, A_t)$. The update is:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(A_t|S_t; \theta) G_t.$$

Interpretation

- If G_t is large (action led to good outcomes), increase the log-probability of the chosen action.
- If G_t is small or negative, decrease the log-probability.

This reinforces beneficial trajectories.

15.5 Reducing Variance: Baselines

REINFORCE is unbiased but high-variance. A baseline $b(s)$ is subtracted without changing the expectation:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(A_t|S_t; \theta) (G_t - b(S_t)).$$

Choosing $b(s) = V_{\pi}(s)$ leads to the **advantage**:

$$A_t = G_t - V(S_t).$$

This reduces variance and is the basis of actor-critic methods.

15.6 REINFORCE Algorithm

For each episode:

1. Generate a trajectory:

$$s_0, a_0, r_1, s_1, a_1, r_2, \dots$$

2. Compute returns:

$$G_t = r_{t+1} + \gamma r_{t+2} + \dots$$

3. For each step:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(a_t|s_t; \theta) G_t.$$

15.7 Neural Policy Parameterization

A typical policy network outputs:

- a softmax distribution for discrete actions:

$$\pi(a|s; \theta) = \text{softmax}(f_{\theta}(s))_a,$$

- a Gaussian distribution for continuous actions:

$$\pi(a|s; \theta) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}(s)).$$

The log-probability $\log \pi(a_t|s_t; \theta)$ is automatically computed via backpropagation.

15.8 Advantages and Limitations

Advantages

- works naturally with continuous action spaces,
- produces smooth, stochastic policies,
- conceptually simple gradient-based update.

Limitations

- very high variance,
- slow convergence,
- requires full episodes (Monte Carlo),
- unstable without baselines.

These limitations motivate actor-critic methods (next chapter).

15.9 Relevance to Trading

Policy gradient methods are useful in trading because:

- actions may be continuous (position size, leverage),
- stochastic policies help exploration in noisy markets,
- policy networks can represent flexible trading strategies,
- variance reduction using baselines stabilizes learning.

However, raw REINFORCE is too high-variance for financial data. Actor-critic methods, PPO, and SAC resolve these issues.

15.10 Summary

- Policy gradients directly optimize $J(\theta)$ rather than action-values.
- REINFORCE uses Monte Carlo estimates and the log-probability gradient.
- Baselines reduce variance and improve learning stability.
- Forms the foundation for actor-critic and advanced deep RL algorithms.

REINFORCE is the simplest policy gradient algorithm and an essential building block toward PPO, A2C, A3C, TRPO, and SAC.

16 Actor–Critic Methods (A2C and A3C)

Policy gradient methods (e.g., REINFORCE) directly optimize a parameterized policy $\pi(a|s; \theta)$, but suffer from extremely high variance and slow learning. Value-based TD methods (e.g., Q-learning) are lower variance but do not directly optimize stochastic policies.

Actor–critic methods combine the benefits of both:

- **Actor:** a policy network that selects actions,
- **Critic:** a value network that estimates $V(s)$ or $Q(s, a)$.

The critic reduces the variance of policy gradients by providing a learned baseline. The actor enables direct optimization of the policy.

This fusion makes actor–critic the foundation of modern deep RL algorithms such as A2C, A3C, PPO, TRPO, and SAC.

16.1 Structure of Actor–Critic

The two components are:

- Policy (actor): $\pi(a|s; \theta)$
- Value function (critic): $V(s; w)$

The actor adjusts θ to maximize expected return, while the critic adjusts w to fit the value function via TD learning.

16.2 Advantage Function

Policy gradients benefit from using the **advantage**:

$$A_t = Q_\pi(S_t, A_t) - V_\pi(S_t).$$

An empirical TD-based estimate is:

$$A_t = r_{t+1} + \gamma V(S_{t+1}; w) - V(S_t; w).$$

The advantage measures how much better an action is compared to the critic’s baseline. Positive advantages increase action probability; negative advantages decrease it.

16.3 Actor Update

Using the policy gradient theorem with an advantage estimate:

$$\theta \leftarrow \theta + \alpha_\theta A_t \nabla_\theta \log \pi(A_t|S_t; \theta).$$

16.4 Critic Update

The critic is trained by minimizing the squared TD error:

$$\delta_t = r_{t+1} + \gamma V(S_{t+1}; w) - V(S_t; w).$$

Gradient descent update:

$$w \leftarrow w + \alpha_w \delta_t \nabla_w V(S_t; w).$$

16.5 A2C: Advantage Actor–Critic

A2C is the synchronous (batched) version of actor–critic:

- multiple workers run environments in parallel,
- collect trajectories,
- compute advantages,
- update shared actor and critic using aggregated gradients.

This reduces variance and improves sample efficiency compared to a single environment.

16.6 A3C: Asynchronous Advantage Actor–Critic

A3C is the asynchronous variant:

- many workers interact with their own copy of the environment,
- each worker maintains its own network copy,
- gradients are pushed asynchronously to a global set of parameters.

Asynchrony decorrelates data without requiring a replay buffer. This made A3C one of the most influential RL algorithms before PPO.

Key Benefits of A3C

- no replay buffer needed,
- massive decorrelation of samples,
- very stable learning,
- excellent performance on continuous control tasks.

16.7 Actor–Critic Algorithm Steps

For each step t :

1. Observe state S_t .
2. Actor samples $A_t \sim \pi(\cdot|S_t; \theta)$.
3. Environment returns reward r_{t+1} and next state S_{t+1} .
4. Critic computes TD error:

$$\delta_t = r_{t+1} + \gamma V(S_{t+1}; w) - V(S_t; w).$$

5. Critic updates:

$$w \leftarrow w + \alpha_w \delta_t \nabla_w V(S_t; w).$$

6. Actor updates:

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi(A_t|S_t; \theta).$$

16.8 Stability Advantages Over REINFORCE

Actor–critic methods:

- use TD learning (lower variance than Monte Carlo),
- update online (no need for full episodes),
- incorporate value baselines,
- learn much faster and more stably.

16.9 Relevance to Trading

Actor–critic methods are useful in trading because:

- continuous action spaces (e.g., position size, leverage),
- extremely noisy returns benefit from variance reduction,
- critic stabilizes learning under market stochasticity,
- actor enables smooth, flexible policies.

A2C/A3C are direct precursors to PPO, which is the most widely used RL algorithm in quantitative finance.

16.10 Summary

- Actor-critic combines value-based TD and policy gradients.
- Actor improves the policy using the advantage function.
- Critic evaluates the policy using TD learning.
- A2C is the synchronized, batched version.
- A3C is the asynchronous, parallelized version.

Actor-critic methods form the conceptual core of modern deep RL, and lead directly to PPO and SAC, the state-of-the-art algorithms for continuous control and trading.

17 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is one of the most widely used and robust policy gradient algorithms in modern reinforcement learning. It improves upon actor-critic methods by ensuring stable and reliable policy updates.

PPO achieves this by constraining how much the policy is allowed to change during each update, preventing catastrophic policy collapse. It is simpler and more practical than Trust Region Policy Optimization (TRPO), while offering comparable performance.

17.1 Motivation

Standard actor-critic suffers from:

- unstable policy updates,
- sensitivity to learning rates,
- large destructive policy changes,
- poor sample efficiency.

PPO introduces a mechanism to limit policy updates using a clipped objective. This ensures:

- small, safe policy improvements,
- reliable convergence,
- good performance across many domains.

17.2 Policy Ratio

PPO compares the new and old policy using the probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}.$$

This measures how much the new policy changes the probability of taking action a_t in state s_t .

17.3 Advantage Function

PPO uses an advantage estimate:

$$A_t = (\text{GAE}) \text{ or } \hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots$$

In practice, PPO commonly uses Generalized Advantage Estimation (GAE), which smooths and reduces variance in the advantage function.

17.4 Clipped Objective

The PPO objective is:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right].$$

Explanation:

- The first term encourages improvement.
- The second (clipped) term prevents large updates.
- The min enforces a conservative update.

ϵ is a small constant (typically 0.1 to 0.2).

17.5 Value Function and Entropy Terms

The full PPO loss includes:

$$L(\theta) = \mathbb{E}_t \left[L^{\text{CLIP}}(\theta) - c_1 (V_\theta(s_t) - V_t^{\text{target}})^2 + c_2 H(\pi_\theta(\cdot|s_t)) \right].$$

Where:

- c_1 weights the value function loss,
- c_2 weights the entropy bonus,
- $H(\pi)$ encourages exploration.

17.6 PPO Algorithm (Simplified)

1. Collect trajectories using the current policy π_θ .
2. Compute advantages $\hat{A}_t$ (using GAE).
3. Compute policy ratios $r_t(\theta)$.
4. Update policy by maximizing the clipped objective:

$$L^{\text{CLIP}}(\theta).$$

5. Update value function via regression.
6. Repeat for several epochs using the same data.

17.7 Why PPO is Stable

PPO enforces:

$$1 - \epsilon \leq r_t(\theta) \leq 1 + \epsilon.$$

This prevents:

- sudden jumps in policy behavior,
- collapsing to deterministic bad actions,
- overly aggressive optimization,
- divergence.

Thus PPO enables multiple gradient steps per batch without destroying the policy.

17.8 Relevance to Trading

PPO is highly suitable for financial trading because:

- continuous actions (position sizes),
- noise-tolerant through GAE,
- stable gradient-based updates,
- consistent performance under stochastic rewards,
- good handling of partial observability,
- safe exploration via entropy regularization.

PPO is one of the most commonly used RL algorithms in trading research due to its robustness and ability to optimize smooth portfolio policies.

17.9 Summary

- PPO is a constrained policy gradient method.
- Uses clipped surrogate objective for stable updates.
- Incorporates value loss and entropy bonuses.
- Supports many update epochs per batch.
- Widely used in continuous-control domains including trading.

PPO builds directly on actor–critic principles, offering a highly stable and practical method for training deep RL agents in large, noisy environments.

18 Soft Actor–Critic (SAC)

Soft Actor–Critic (SAC) is a state-of-the-art off-policy actor–critic algorithm designed for continuous action spaces. It achieves outstanding stability and sample efficiency by optimizing not only expected return, but also the *entropy* of the policy.

SAC encourages exploration and avoids premature convergence to suboptimal deterministic policies—properties highly useful in noisy domains such as financial markets.

18.1 Motivation

Traditional actor–critic methods (A2C, A3C, PPO) optimize expected return:

$$\max_{\theta} \mathbb{E}[G_t].$$

However:

- deterministic policies risk getting stuck,
- insufficient exploration leads to poor solutions,
- unstable training arises from sharp policy updates,
- noisy rewards (e.g., asset returns) create brittle behavior.

SAC improves robustness by maximizing:

$$\text{return} + \text{entropy}.$$

18.2 Maximum Entropy RL Objective

SAC maximizes the stochastic policy objective:

$$J(\pi) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \pi} \left[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right],$$

where:

$$\mathcal{H}(\pi(\cdot|s)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a|s)]$$

is the entropy, and α controls the tradeoff between reward and exploration.

Interpretation

- The agent seeks high reward *and* high randomness.
- Prevents collapse into premature deterministic behavior.
- Ensures persistent exploration even in late training.

18.3 SAC Architecture

SAC maintains multiple networks:

- policy network (actor) $\pi_\theta(a|s)$,
- two Q-networks (critics) $Q_{\phi_1}(s, a)$ and $Q_{\phi_2}(s, a)$,
- target Q-networks for stability.

Why two Q-networks?

Like Double DQN, SAC reduces overestimation by using:

$$\min(Q_{\phi_1}(s, a), Q_{\phi_2}(s, a)).$$

18.4 Critic Update

For a transition (s, a, r, s') , SAC targets are:

$$y = r + \gamma \left(\min_j Q_{\phi_j}(s', a') - \alpha \log \pi_\theta(a'|s') \right),$$

with a' sampled from the current policy.

Critics minimize:

$$L(\phi_j) = \frac{1}{2} \mathbb{E} \left[Q_{\phi_j}(s, a) - y \right]^2.$$

18.5 Actor Update

The policy update minimizes the KL-like objective:

$$\theta \leftarrow \theta - \alpha_\theta \mathbb{E} \left[\alpha \log \pi_\theta(a|s) - \min_j Q_{\phi_j}(s, a) \right],$$

where $a \sim \pi_\theta(\cdot|s)$.

Interpretation:

- Maximize Q-values.
- Maximize entropy (high randomness).

The result is a policy that balances exploitation and structured randomness.

18.6 Automatic Entropy Adjustment

A key innovation in SAC is automatically adjusting the entropy temperature α :

$$\alpha \leftarrow \alpha - \lambda_\alpha \mathbb{E} \left[\log \pi_\theta(a|s) + \mathcal{H}_{\text{target}} \right].$$

This ensures:

- desired exploration level is maintained,
- no need for manual tuning of α ,
- stable behavior across environments.

18.7 Why SAC Works Extremely Well

- **Off-policy:** uses replay buffer $\rightarrow$ sample efficient.
- **Double critics:** eliminates value overestimation.
- **Entropy term:** ensures strong exploration and robustness.
- **Stable targets:** with target critics.
- **Smooth policies:** suitable for continuous domains.

18.8 Relevance to Trading

SAC is one of the best RL algorithms for financial markets because:

- rewards are noisy and stochastic,
- exploration must be persistent (market shifts),
- continuous position sizing is often required,

- noise-robust Q-estimation improves stability,
- entropy helps avoid overconfident trades,
- off-policy learning increases sample efficiency.

SAC is commonly used for:

- portfolio allocation (continuous action),
- optimal execution,
- market-making,
- risk-sensitive trading agents.

18.9 Summary

- SAC maximizes reward + entropy for robustness.
- Two critics reduce overestimation.
- Policy outputs continuous distributions.
- Automatic entropy tuning stabilizes training.
- Off-policy design enables efficient data reuse.

Soft Actor–Critic is one of the best-performing RL algorithms in practical, stochastic environments and is highly applicable to modern quantitative trading research.

19 Designing a Reinforcement Learning Environment for Trading

Implementing RL for trading requires constructing a custom environment that conforms to the **gym**-style interface used by most RL libraries. The environment must define:

- the **state space** (market observations),
- the **action space** (trading decisions),
- the **reward function** (profit, risk-adjusted performance),
- **episode structure** (start & end of trading periods),
- **transaction costs and slippage**,
- **portfolio evolution**.

The quality of these design choices directly determines whether the RL agent will behave realistically and learn profitable strategies.

19.1 State Space Design

The state s_t represents information available to the agent at time t . A good trading state includes:

- **price-based features:**

$$P_t, P_{t-1}, \dots, \Delta P, \text{ log-returns}$$

- **technical indicators:**

$$\text{MA, EMA, RSI, MACD, ATR, Volatility}$$

- **volume features:**

$$V_t, \Delta V, OBV, \text{ liquidity measures}$$

- **portfolio variables:**

$$\text{current position size, holdings, cash}$$

- **risk metrics:**

$$\text{drawdown, rolling volatility, Sharpe estimates}$$

Feature normalization is essential:

$$x'_t = \frac{x_t - \mu}{\sigma}.$$

19.2 Action Space

Actions represent trading decisions. They can be:

- **Discrete:**

$$a_t \in \{\text{Buy, Sell, Hold}\}$$

commonly used with DQN.

- **Continuous:**

$$a_t \in [-1, 1]$$

where -1 is full short, $+1$ is full long, used with PPO or SAC.

Continuous actions allow:

- position sizing,
- partial buys/sells,
- risk-adjusted exposure control.

19.3 Portfolio and Position Dynamics

At each step:

$$\text{position}_t = f(\text{position}_{t-1}, a_t).$$

Portfolio value evolves as:

$$\text{PV}_{t+1} = \text{PV}_t + \text{position}_t \cdot (P_{t+1} - P_t) - \text{transaction costs}.$$

Transaction cost model:

$$\text{Cost}_t = c \cdot |\Delta \text{position}_t|,$$

where c is the rate (e.g., 0.1% to 0.3%).

Slippage can be modeled as:

$$\text{Slippage} = \kappa \cdot |\Delta \text{position}_t| \cdot \text{Volatility}_t.$$

19.4 Reward Function Design

Common reward choices:

- **Step return:**

$$r_t = \text{PV}_{t+1} - \text{PV}_t.$$

- **Return percentage:**

$$r_t = \frac{\text{PV}_{t+1} - \text{PV}_t}{\text{PV}_t}.$$

- **Risk-adjusted reward:**

$$r_t = \frac{R_t}{\sigma_t},$$

approximating the Sharpe ratio.

- **Penalizing drawdown:**

$$r_t = R_t - \lambda \cdot \text{Drawdown}_t.$$

Reward Design Guidelines

- Rewards must include transaction costs.
- Reward scaling is important for stable training.
- Avoid raw P&L because it creates high variance.

Risk-aware rewards (Sharpe-like) are recommended for noisy markets.

19.5 Episode Structure

Episodes may represent:

- fixed-length windows (e.g., 200 steps),
- one full trading year,
- a sliding window over historical data,
- or full dataset rollouts.

Termination conditions:

- end of data,
- margin call (PV drops below threshold),
- excessive drawdown,
- fixed number of steps.

19.6 Gym-style Interface

A typical environment implements:

- `reset()`: returns initial state s_0 ,
- `step(a)`: returns $(s_{t+1}, r_t, \text{done}, \text{info})$.

State transitions:

$$s_{t+1} = f(s_t, a_t, \text{market data})$$

19.7 Training Loop Integration

Replay buffers (for off-policy methods like SAC/DQN):

$$\mathcal{D} \leftarrow (s_t, a_t, r_t, s_{t+1}).$$

On-policy methods (PPO, A2C):

- trajectories are collected,
- advantages are computed,
- policy and value networks updated over multiple epochs.

19.8 Common Pitfalls

- Using unnormalized features.
- Reward functions without transaction costs.
- Overfitting to historical data (illusory profits).
- Using raw price as a feature (non-stationary).
- Not randomizing train/validation periods.

19.9 Practical Trading Considerations

Transaction costs are essential. Failing to include them leads to unrealistic overtrading.

Data leakage / lookahead bias:

- ensure indicators use only past data,
- normalize using rolling statistics instead of global ones,
- avoid future information in features.

Walk-forward validation:

- train on early period,
- validate on next segment,
- roll forward.

19.10 Relevance to RL Trading Agents

A well-designed environment ensures:

- stable agent behavior,
- realistic market interaction,
- safe trading dynamics,
- robust generalization to unseen data.

Environment design is often more important than the choice of RL algorithm.

19.11 Summary

- The trading environment defines states, actions, rewards, and transitions.
- Proper feature engineering and normalization are crucial.
- Rewards must reflect P&L, risk, and trading costs.
- Episode design affects learning stability.
- Common pitfalls include leakage, unrealistic rewards, and overtrading.

A carefully constructed RL trading environment is the foundation for building profitable and robust trading agents using PPO, SAC, and other advanced algorithms.